

```

#include <SoftwareSerial.h>
#include <TinyGPS++.h>
#define RXPin 2
#define TXPin 3
#define GPSTBaud 4800
#define ConsoleBaud 115200

// The serial connection to the GPS device
SoftwareSerial ss(RXPin, TXPin);

// The TinyGPS++ object
TinyGPSPlus gps;
unsigned long lastUpdateTime = 0;

#define EIFFEL_LAT 48.85823
#define EIFFEL_LNG 2.29438

/* This example shows a basic framework for how you might
   use course and distance to guide a person (or a drone)
   to a destination. This destination is the Eiffel Tower.
   Change it as required.

   The easiest way to get the lat/long coordinate is to
   right-click the destination in Google Maps (maps.google.com),
   and choose "What's here?". This puts the exact values in the
   search box.
*/

void setup()
{
  Serial.begin(ConsoleBaud);
  ss.begin(GPSTBaud);

  Serial.println("GPS Example 3");
  Serial.println("A Not-so-comprehensive Guidance System");
  Serial.println("by Mikal Hart");
  Serial.println();
}

void loop()
{
  // If any characters have arrived from the GPS,
  // send them to the TinyGPS++ object
  while (ss.available() > 0)
    gps.encode(ss.read());

  // Every 5 seconds, do an update.
  if (millis() - lastUpdateTime >= 5000)
  {
    lastUpdateTime = millis();
    Serial.println();

    // Establish our current status
    double distanceToDestination = TinyGPSPlus::distanceBetween(
      gps.location.lat(), gps.location.lng(), EIFFEL_LAT, EIFFEL_LNG);
    double courseToDestination = TinyGPSPlus::courseTo(
      gps.location.lat(), gps.location.lng(), EIFFEL_LAT, EIFFEL_LNG);
    const char *directionToDestination =
      TinyGPSPlus::cardinal(courseToDestination);
  }
}

```

```

    int courseChangeNeeded = (int)(360 + courseToDestination - gps.course.deg()) %
360;

    // debug
    Serial.print("DEBUG: Course2Dest: ");
    Serial.print(courseToDestination);
    Serial.print("  CurCourse: ");
    Serial.print(gps.course.deg());
    Serial.print("  Dir2Dest: ");
    Serial.print(directionToDestination);
    Serial.print("  RelCourse: ");
    Serial.print(courseChangeNeeded);
    Serial.print("  CurSpd: ");
    Serial.println(gps.speed.kmph());

    // Within 20 meters of destination? We're here!
    if (distanceToDestination <= 20.0)
    {
        Serial.println("CONGRATULATIONS: You've arrived!");
        exit(1);
    }

    Serial.print("DISTANCE: ");
    Serial.print(distanceToDestination);
    Serial.println(" meters to go.");
    Serial.print("INSTRUCTION: ");

    // Standing still? Just indicate which direction to go.
    if (gps.speed.kmph() < 2.0)
    {
        Serial.print("Head ");
        Serial.print(directionToDestination);
        Serial.println(".");
        return;
    }

    if (courseChangeNeeded >= 345 || courseChangeNeeded < 15)
        Serial.println("Keep on straight ahead!");
    else if (courseChangeNeeded >= 315 && courseChangeNeeded < 345)
        Serial.println("Veer slightly to the left.");
    else if (courseChangeNeeded >= 15 && courseChangeNeeded < 45)
        Serial.println("Veer slightly to the right.");
    else if (courseChangeNeeded >= 255 && courseChangeNeeded < 315)
        Serial.println("Turn to the left.");
    else if (courseChangeNeeded >= 45 && courseChangeNeeded < 105)
        Serial.println("Turn to the right.");
    else
        Serial.println("Turn completely around.");
}
}

```